

开源软件基础结课论文

对 PostgreSQL 的分析

作者姓名：王虹景，李霄冲

指导教师：任志磊

学位类别：工学学士

专 业：软件工程

学院（系）：大连理工大学

2026 年 2 月

Analysis of PostgreSQL

**A thesis submitted to
University of Chinese Academy of Sciences
in partial fulfillment of the requirement
for the degree of
Bachelor of Engineering
in Software Engineering**

By

Wang Hongjing, Li Xiaochong

Supervisor: Ren Zhilei

Dalian University of Technology

February, 2026

摘 要

本项目围绕 PostgreSQL master 分支（2022-2025）的提交历史，构建“提交行为统计—Bug 修复识别—静态结构演化—动态执行追踪—形式化验证”的多层分析链路，系统刻画 Bug 修复行为规律与代码演化特征，并对关键修复进行可验证性证明。在提交层面，基于规则化标注与时间/作者/模块统计，发现修复节奏与发布周期相关，高风险模块（storage/replication/WAL 等）长期高占比，且核心开发者承担关键模块的修复工作。在静态结构层面，Bug 修复呈现出一致的结构化演化模式：前置条件显式化、控制流分裂、错误路径显式化与条件语义增强等，体现防御式编程的系统性强化。在动态分析层面，通过 PySnooper 追踪 PostgreSQL Python 工具脚本，定位出配置同步脚本的“静默逻辑失效”，并通过回归测试验证修复有效性，建立了从异常发现到验证的闭环流程。在形式化验证层面，使用 Z3 对该修复进行约束建模，证明原逻辑存在反例（SAT），修复后逻辑在规约域内无反例（UNSAT），从而实现“证明修复有效”的强结论。整体而言，本研究以统计分析、结构分析、动态追踪与形式化建模互证，为理解大型系统的 Bug 修复规律与提升修复可信度提供了可复用的方法框架。

关键词：PostgreSQL，提交历史分析，Bug 修复识别，代码结构演化，动态追踪，形式化验证

Abstract

This thesis builds a multi-layer analysis pipeline over the PostgreSQL master branch commit history (2022–2025), integrating commit-level statistics, bug-fix identification, static structural evolution analysis, dynamic execution tracing, and formal verification. At the commit level, using rule-based labeling and statistics by time, author, and subsystem, we observe that fix activity correlates with release cycles; high-risk subsystems (e.g., storage, replication, and WAL) consistently account for a large share of fixes, and core developers contribute disproportionately to critical-module maintenance. At the static level, bug fixes exhibit recurring structural evolution patterns, including explicit preconditions, control-flow splitting, explicit error paths, and enriched conditional semantics, reflecting systematic strengthening of defensive programming. At the dynamic level, we trace PostgreSQL Python utility scripts with PySnooper to pinpoint a silent logic failure in a configuration synchronization script, validate the fix via regression tests, and close the loop from anomaly discovery to verification. Finally, we model the fix in Z3: the original logic admits counterexamples (SAT), whereas the repaired logic has no counterexample within the specification domain (UNSAT), yielding a strong, proof-backed validity claim. Overall, the proposed framework triangulates statistical analysis, structural analysis, dynamic tracing, and formal modeling to provide reusable methods for understanding bug-fix behaviors in large systems and improving the trustworthiness of fixes.

Keywords: PostgreSQL, commit history mining, bug-fix identification, structural evolution, dynamic tracing, formal verification

目 录

第 1 章 绪论	1
1.1 研究背景与问题定义	1
1.2 研究目标与贡献	1
1.3 研究方法与技术路线	1
1.4 论文结构	1
第 2 章 数据集构建与 Bug 修复识别	2
2.1 PostgreSQL 提交数据范围与来源 (2022–2025)	2
2.2 Bug 修复提交识别规则 (关键词/结构/测试信号)	2
2.3 标注流程与质量控制 (Y/M/N 与复核机制)	2
2.4 描述性统计 (时间、作者、模块、规模)	3
第 3 章 Bug 修复结构演化分析 (静态)	4
3.1 AST 与 libcst 分析方法	4
3.2 典型修复模式: 前置条件显式化与错误路径显式化	4
3.3 控制流与条件复杂度变化	5
3.4 静态分析小结	6
第 4 章 动态执行轨迹与案例复现	7
4.1 动态分析对象与实验环境	7
4.2 generate_editorconfig.py 的异常定位 (Silent Failure)	7
4.3 修复策略与回归测试结果	8
4.4 动态证据对静态结论的补充价值	9
4.5 本章小结	9
第 5 章 形式化建模与修复正确性验证	10
5.1 Z3 建模对象与状态抽象	10
5.2 修复前模型: 反例可满足性 (SAT)	10
5.3 修复后模型: 反例不可满足性 (UNSAT)	10
5.4 从“测试发现”到“证明正确”的方法提升	11
第 6 章 综合讨论	12
6.1 三类证据链 (静态–动态–形式化) 的互证关系	12
6.2 高风险模块治理启示 (storage/replication/WAL)	12
6.3 方法局限与有效性威胁	12

第 7 章 结论与未来工作	13
7.1 主要研究结论	13
7.2 工程实践建议	13
7.3 后续工作（自动检测、自动修复、CI 集成）	13
附录 A 大学学位论文撰写要求	14
A.1 论文无附录者无需附录部分	14
A.2 测试公式编号 $\Lambda, \lambda, \theta, \bar{\Lambda}, \sqrt{S_{NN}}$	14
A.3 测试生僻字	15
参考文献	16
致谢	16
作者简历及攻读学位期间发表的学术论文与研究成果	17

符号列表

字符

Symbol	Description	Unit
R	the gas constant	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_v	specific heat capacity at constant volume	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
C_p	specific heat capacity at constant pressure	$\text{m}^2 \cdot \text{s}^{-2} \cdot \text{K}^{-1}$
E	specific total energy	$\text{m}^2 \cdot \text{s}^{-2}$
e	specific internal energy	$\text{m}^2 \cdot \text{s}^{-2}$
h_T	specific total enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
h	specific enthalpy	$\text{m}^2 \cdot \text{s}^{-2}$
k	thermal conductivity	$\text{kg} \cdot \text{m} \cdot \text{s}^{-3} \cdot \text{K}^{-1}$
S_{ij}	deviatoric stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
τ_{ij}	viscous stress tensor	$\text{kg} \cdot \text{m}^{-1} \cdot \text{s}^{-2}$
δ_{ij}	Kronecker tensor	1
I_{ij}	identity tensor	1

算子

Symbol	Description
Δ	difference
∇	gradient operator
$\delta^\pm$	upwind-biased interpolation scheme

缩写

CFD	Computational Fluid Dynamics
CFL	Courant-Friedrichs-Lewy
EOS	Equation of State

JWL	Jones-Wilkins-Lee
WENO	Weighted Essentially Non-oscillatory
ZND	Zel'dovich-von Neumann-Doering

第 1 章 绪论

1.1 研究背景与问题定义

本文聚焦 PostgreSQL 在 2022–2025 年间的提交演化，关注其中具有代表性的 Bug 修复行为。研究问题是：在真实工程中，Bug 修复是否存在可归纳的结构模式，以及这些模式能否通过静态、动态和形式化三类证据被一致支持。

1.2 研究目标与贡献

本文的目标是构建一个可复现的数据与分析流程，并围绕“识别–分析–验证”形成完整证据链。主要贡献包括：

- 构建 PostgreSQL Bug 修复样本集，并建立可执行的标注规范；
- 提炼修复结构演化规律，分析控制流与条件复杂度变化；
- 在典型案例上结合动态执行轨迹与形式化约束验证修复正确性。

1.3 研究方法与技术路线

技术路线分为三层：

1. 数据层：提交抓取、规则识别、人工复核与统计分析；
2. 证据层：AST/libcst 静态分析与运行轨迹复现；
3. 验证层：基于 Z3 的状态抽象与可满足性检验。

1.4 论文结构

全文共七章：第二章介绍数据集和识别流程；第三章给出静态结构演化结果；第四章给出动态案例复现；第五章进行形式化验证；第六章综合讨论三类证据及治理启示；第七章总结并展望。

第 2 章 数据集构建与 Bug 修复识别

2.1 PostgreSQL 提交数据范围与来源（2022–2025）

本研究的数据来自仓库中已整理的提交级 CSV 文件与汇总报告，时间范围覆盖 2022–2025 年。核心数据资产包括：

- `raw_enhanced.csv`：提交哈希、作者、时间、提交信息与信息密度；
- `final_summary_report.csv`：年度修复提交规模与代码 churn 汇总；
- `bug_fixes_detailed_2023.csv`：2023 年细粒度增删行与模块字段。

依据 `final_summary_report.csv`，2022–2025 年的修复提交数量分别为 692、619、820、829；对应年度 churn 分别为 41610、30477、34469、36852 行。四年合计修复提交 2960 次，显示 2024–2025 年活跃度显著提升。

2.2 Bug 修复提交识别规则（关键词/结构/测试信号）

本研究采用成员二制定的三类信号联合识别策略：

1. **关键词信号**：优先使用强关键词（`fix/bug/crash/segfault/assert/regress/leak`），弱关键词（`check/boundary/timeout/retry`）仅作为候选补充；
2. **结构信号**：检查是否出现条件与边界检查新增、错误处理 API（`ereport/elog`）强化、锁与资源管理收紧；
3. **测试信号**：观察是否同步修改 `src/test/**` 或 `contrib/*/sqllexpected`，以及提交信息是否提及 `regression/backpatch/CVE`。

在 `raw_enhanced.csv` 中进行消息级统计，`fix` 相关关键词命中 2422 次，`error` 412 次，`assert` 157 次，`regression` 107 次，说明“显式修复语义”在提交文本中占主导地位。

2.3 标注流程与质量控制（Y/M/N 与复核机制）

标注流程遵循“规则筛查 + 人工复核”两阶段：

- 第一步：通过 `git log -grep` 按关键词生成候选集合；
- 第二步：对候选逐条 `git show -U3`，按 **Y/M/N** 标注（明确修复/疑似修复/非修复）；

- 第三步: 对 storage/replication/WAL/locking 等高风险模块执行二人复核, 保留冲突裁决记录。

标注字段建议包含 is_bugfix, signals, modules, reason, loc_add, loc_del, date, author, 以便后续做时间、作者、模块和规模四维交叉分析。

2.4 描述性统计（时间、作者、模块、规模）

时间分布

从 raw_enhanced.csv 统计得到: 2022–2025 年样本量分别为 724、644、840、847（总计 3055 条）; 月度峰值集中在 2025-04（109 条）与 2024-04（104 条）, 与版本开发活跃窗口基本一致。

作者分布

Top-3 作者分别为 Tom Lane（497）、Michael Paquier（463）、Peter Eisentraut（309）, 前三位合计约占 41.2%, 显示核心维护者在修复活动中承担了主要工作量。

模块与规模

在 677 条静态分析样本（static_analysis_diffs）中, 路径聚合显示 replication 与 storage 各 45 条, wal 相关 13 条, 验证了高风险底层模块的持续修复压力。以 2023 年细粒度样本为例, 中位新增/删除行为为 7/3 行, 新增行 P90 约为 79 行, 说明多数修复为“小步快修”, 但仍存在少量大规模补丁。

本章得到的数据与标注结果, 为第三章静态结构分析和第四、五章案例验证提供了统一样本基础。

第 3 章 Bug 修复结构演化分析（静态）

3.1 AST 与 libcst 分析方法

本章目标是回答“修复代码到底在结构上改变了什么”。为避免仅停留在文本 diff 层，本研究采用“diff 抽取 → 结构特征提取 → 模式归纳”的三步流程，并与仓库已有产物对齐：

1. 数据准备：使用 `static_analyze/diff_extract.py` 生成 677 份补丁差异文件 (`static_analysis_diffs/*.diff`)；
2. 结构提取：基于 AST/语义规则识别 if/else、谓词组合、错误路径与返回路径变化，输出到 `ast_logic_analysis_report.csv`；
3. 语义归纳：结合 `bug_fix_structure_feature.md` 与样本 diff，统一抽象为跨案例可迁移的修复模式。

需要强调的是，PostgreSQL 代码中宏与类型别名密集，直接解析 C AST 容易出现语法中断；因此分析中采取了预处理脱敏策略，先保证可解析性，再统计节点差异。该步骤并不改变修复语义，只降低语法噪声对结构统计的干扰。

样本范围与覆盖

样本覆盖 2022–2025 年，且以中等规模补丁为主，以减少“纯格式化”与“大规模重构”带来的偏差。路径聚合统计显示 677 条样本中，`storage` 与 `replication` 各 45 条，`executor` 31 条，`catalog` 24 条，`planner` 22 条，`wal` 13 条。该分布与前文“高风险模块长期活跃”的观察一致。

3.2 典型修复模式：前置条件显式化与错误路径显式化

通过对 677 条样本进行结构归并，本研究识别出两类最稳定、最具工程解释力的模式。

模式 A：前置条件显式化（Precondition Explicitness）

该模式的核心是把“默认成立”的假设改写成“代码可检查”的约束，典型表现包括：

- 新增 NULL/边界/状态校验；

- 将主逻辑包裹入 `guard`，非法输入提前返回；
- 将隐含前提从注释语义提升为可执行条件。

结构上，这类修复会带来 `if` 节点数增加、`return` 路径增加、入口条件变强。它的工程价值在于：将“运行后才暴露的问题”前移为“入口可拒绝的问题”，降低错误传播深度。

模式 B：错误路径显式化（Error Path Materialization）

该模式对应“`silent failure` 治理”，表现为：

- 从隐式失败切换为显式错误分支；
- 新增 `ereport/elog` 或错误码返回；
- 正常路径与异常路径分离，便于测试与审计。

从控制流视角看，这一变化等价于“把原本不可见的失败状态加入状态机”。在审计实践中，这类改动可显著提升问题可诊断性，也能为后续形式化规约提供清晰的失败语义。

复合模式：Logic + Memory

仓库静态报告指出，逻辑分支缺陷常与内存/资源问题联动出现。具体而言，条件遗漏会放大到资源生命周期错误（如释放时机错误、越界触发窗口扩大）。因此，修复并非“局部补丁”，而是对条件空间与副作用边界的共同收紧。

3.3 控制流与条件复杂度变化

本节从“控制流图变化”和“谓词复杂度变化”两个维度细化静态证据。

控制流图（CFG）变化趋势

修复前，部分样本呈单路径直通执行；修复后常出现以下变化：

1. 新增入口 `guard` 分支，非法状态快速退出；
2. 将异常流程独立为错误路径，避免与正常路径混杂；
3. `return` 点位增加，但路径语义更可解释。

虽然 CFG 节点数可能上升，但这种“复杂度上升”在工程上通常是正向复杂度：系统用更明确的路径表达不变量，而非依赖隐式假设。

条件表达式复杂度变化

在谓词层面，修复常由“单条件”演化为“复合条件”：

- 从 $x > 0$ 扩展为 $x \neq \text{NULL} \ \&\& \ x > 0$;
- 从状态单点判断扩展为状态 + 资源 + 上下文联合约束;
- 引入 AND/OR 组合后，路径判定边界更清晰。

这种演化意味着约束空间被细分，错误输入更难穿透到核心逻辑区，是“防御式编程增强”的直接结构证据。

可维护性影响讨论

修复后的代码行数与分支数可能增加，但维护成本并不一定恶化。相反，当条件语义、失败路径、日志语义三者对齐时，后续维护者可更快定位问题并进行局部验证。换言之，**可解释性提升**抵消了部分表面复杂度。

3.4 静态分析小结

本章结论可概括为三条：

1. PostgreSQL 修复呈现稳定的结构演化方向：**约束前移、错误显式化、谓词增强**；
2. 高风险模块（storage/replication/WAL）中此类模式更集中，说明关键路径治理依赖结构性修复；
3. 静态分析不仅解释“代码改了哪里”，更输出了可被动态复现与形式化验证复用的条件框架。

因此，静态证据在本研究中承担“模式提炼与规约生成”的基础角色，为第四章运行时验证与第五章证明级验证提供可转译输入。

第 4 章 动态执行轨迹与案例复现

4.1 动态分析对象与实验环境

如果第三章回答的是“结构怎么变”，那么本章回答的是“这些结构变化在运行时是否真正改变了系统行为”。为此，本章选取仓库中最具代表性的 Python 工具链案例进行动态追踪与回归复现，重点对象如下：

- 案例 A: `generate_editorconfig.py` (2025)，负责将 `.gitattributes` 规则同步到 `.editorconfig`；
- 案例 B: `generate_unaccent_rules.py` (2022)，用于规则文件生成与参数验证。

实验材料来自 `dynamic_analyze` 目录，包括：`da_editorconfig_sync.log`、`da_unaccent_rules.log`、`test_validation.log`、`test_validation_fixed.log`、`test_editorconfig_sync.py` 与修复脚本 `generate_editorconfig_fixed.py`。分析工具链为 `pysnooper` + `pytest` + `subprocess`，用于同时捕捉语句级轨迹和测试级可观测行为。

环境与可重复性说明

日志显示实验环境为 Windows + Python 3.10。路径与子进程调用记录表明：脚本可以稳定启动、路径解析正常、`stdout/stderr` 可被捕获。该前提非常关键，因为它排除了“环境不兼容导致失败”的干扰，使我们能够聚焦逻辑本身。

4.2 `generate_editorconfig.py` 的异常定位 (Silent Failure)

该案例暴露的是一种比崩溃更隐蔽的问题：**返回成功但语义失败**。

现象层证据

在 `test_validation.log` 中，测试集共 2 项，其中 1 项失败。失败断言为：

```
assert "[*.py]" in content
```

这意味着脚本虽然执行成功 (`returncode=0`)，却未把 Python 规则正确写入输出配置，表现为“状态看似健康、结果实则错误”。

轨迹层证据

结合 `da_editorconfig_sync.log`，可观察到 `subprocess` 调用与输出捕获流程均正常，问题并非出在进程调度或 I/O。进一步对照测试断言与脚本逻辑可定位到：属性映射路径未完整覆盖 Python 分支，导致满足 `filter=indent` 的输入回退为全局默认路径。

为何属于 Silent Failure

该缺陷满足 `silent failure` 的三个判据：

1. 程序无崩溃、无异常退出；
2. 返回码显示成功，容易被 CI 误判为通过；
3. 业务语义目标（Python 专用规则）未达成。

这类问题仅靠“进程是否成功”无法发现，必须结合语义断言或轨迹比对。

4.3 修复策略与回归测试结果

针对缺陷根因，仓库提供了修复版 `generate_editorconfig_fixed.py`。修复思想不是“大改逻辑”，而是补全映射状态机：在原有 C/H 分支之外新增 Python 分支，并保留默认兜底。

回归测试结果

`test_validation_fixed.log` 显示同一测试集从“1 fail + 1 pass”提升为“2 pass”。关键改进点如下：

- 语义断言 `/*.py` 成立，说明目标规则成功落盘；
- 未引入新增失败用例，回归稳定；
- 样本日志显示耗时由 0.11s 下降到 0.06s（仅作为样本观测，不作性能结论外推）。

修复有效性的工程解释

本次修复的有效性并不只体现在“测试变绿”，更体现在输出语义被恢复：从“仅全局配置”变为“全局 + Python 特化配置”。这说明补丁真正改变了业务行为，而非仅规避触发条件。

4.4 动态证据对静态结论的补充价值

动态分析对第三章静态结论提供了四个层面的补充：

1. **把结构模式映射到运行结果**：静态上“错误路径显式化”，动态上体现为测试可观察的失败/成功切换；
2. **把条件增强映射到行为约束**：静态上“新增分支”，动态上体现为 Python 输入不再回退到默认分支；
3. **提供反例最小化场景**：FileType=Python, Attribute=indent 直接成为第五章 SMT 规约输入；
4. **形成证据闭环**：日志轨迹（过程）+ 测试断言（结果）共同支撑“修复确实生效”。

对案例 B 的补充说明

generate_unaccent_rules.py 的日志则体现了另一类动态价值：参数缺失时系统给出明确错误信息并安全退出，说明其错误路径已较为显式化。这与案例 A 形成对照，进一步说明“是否显式建模错误路径”会直接决定缺陷可见性。

4.5 本章小结

本章通过动态复现证明：

- 静态结构变化可以转化为可观测的运行时行为差异；
- silent failure 必须依赖语义断言与轨迹协同才能稳定识别；
- 修复后的行为改进可通过回归测试闭环确认，并可直接转译为形式化规约。

因此，动态分析在本研究中承担“从结构推断走向行为验证”的关键桥梁角色。

第 5 章 形式化建模与修复正确性验证

5.1 Z3 建模对象与状态抽象

本章基于成员五提交的 `verify_config_logic.py` 与 `verify_fix.py`，将第四章动态复现得到的缺陷抽象为可判定约束问题。模型变量定义如下：

- **FileType**: 文件类型 (0=Unknown, 1=C, 2=H, 3=Python)；
- **Attribute**: 属性类型 (0=None, 1=FilterIndent, 2=Other)；
- **OutputAction**: 动作 (0=DoNothing, 1=GenCCConfig, 2=GenPyConfig)。

验证规约 (Specification) 统一设定为：当 `FileType=3` 且 `Attribute=1` 时，系统必须满足 `OutputAction=2`。若存在满足输入条件但不满足输出条件的模型，即判定逻辑存在漏洞。

5.2 修复前模型：反例可满足性 (SAT)

在修复前模型中，核心逻辑为：C/H 文件输出 C 配置，其余类型默认 DoNothing。该实现天然缺失 Python 分支。将“违规条件”加入求解器后，理论上可得到 SAT：

存在一组输入使得 $\text{FileType}=3 \wedge \text{Attribute}=1 \wedge \text{OutputAction} \neq 2$ 成立。

这与第四章的动态反例一致：Python 文件在需要缩进规则时未被正确处理，形成“成功返回 + 语义失败”的静默缺陷。

5.3 修复后模型：反例不可满足性 (UNSAT)

修复后模型显式加入 `FileType=3 -> OutputAction=2` 的分支约束，逻辑结构变为 C/H 分支、Python 分支与默认分支三级映射。此时对同一违规条件求解，理论结果为 UNSAT：

在修复后约束系统中，不存在满足违规条件的输入赋值。

因此，修复后模型不仅“通过测试”，更在给定抽象域内达成“无反例”证明。

5.4 从“测试发现”到“证明正确”的方法提升

本章体现的方法提升可概括为三点：

1. **证据层级提升**：从经验性测试（样本输入）提升到约束空间证明（输入域）；
2. **可解释性提升**：反例由具体日志提升为可读的逻辑赋值（变量取值可复现）；
3. **闭环完整性提升**：形成“动态复现 Bug $\rightarrow$ 形式化反例 SAT $\rightarrow$ 修复后 UNSAT”的可追踪链路。

需要说明的是，形式化结论依赖于抽象边界：若后续引入新属性类型或更复杂路径规则，应同步扩展状态空间与规约集合，以维持证明有效性。

第 6 章 综合讨论

6.1 三类证据链（静态-动态-形式化）的互证关系

静态分析回答“改了什么结构”，动态分析回答“运行时发生了什么”，形式化分析回答“是否仍存在可达反例”。三者构成互补证据链，提升结论可信度。

6.2 高风险模块治理启示（storage/replication/WAL）

针对高风险模块（storage/replication/WAL），建议建立分层治理机制：高频提交监测、关键路径断言增强、回归用例优先配置以及形式化检查前置。

6.3 方法局限与有效性威胁

本文方法存在若干局限：样本时间窗口有限、标注仍受人工判断影响、形式化模型存在抽象损失。外部有效性方面，不同项目的开发流程差异可能影响结论迁移。

第 7 章 结论与未来工作

7.1 主要研究结论

本文围绕 PostgreSQL Bug 修复演化给出系统研究：构建数据集与识别规则，提炼静态修复模式，完成动态复现，并通过 Z3 提供修复正确性验证。

7.2 工程实践建议

建议在工程流程中落地三项实践：

- 将修复识别与模块风险统计纳入例行质量看板；
- 将动态回归与关键路径日志作为发布前强制检查；
- 对高风险逻辑引入轻量级形式化验证。

7.3 后续工作（自动检测、自动修复、CI 集成）

后续可从三方面扩展：基于历史修复模式训练自动检测器；探索面向条件与错误路径的自动修复策略；将静态、动态和形式化检查进一步集成到 CI 流水线。

附录 A 大学学位论文撰写要求

学位论文是研究生科研工作成果的集中体现，是评判学位申请者学术水平、授予其学位的主要依据，是科研领域重要的文献资料。根据《科学技术报告、学位论文和学术论文的编写格式》（GB/T 7713-1987）、《学位论文编写规则》（GB/T 7713.1-2006）和《文后参考文献著录规则》（GB7714—87）等国家有关标准与实际情况，特制订本规定。

A.1 论文无附录者无需附录部分

A.2 测试公式编号 $\Lambda, \lambda, \theta, \bar{\Lambda}, \sqrt{S_{NN}}$

$$\begin{cases} \frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{V}) = 0 \\ \frac{\partial(\rho \mathbf{V})}{\partial t} + \nabla \cdot (\rho \mathbf{V} \mathbf{V}) = \nabla \cdot \boldsymbol{\sigma} \\ \frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\rho E \mathbf{V}) = \nabla \cdot (k \nabla T) + \nabla \cdot (\boldsymbol{\sigma} \cdot \mathbf{V}) \end{cases} \quad \dots (A.1)$$

$$\frac{\partial}{\partial t} \int_{\Omega} u \, d\Omega + \int_S \mathbf{n} \cdot (u \mathbf{V}) \, dS = \dot{\phi} \quad \dots (A.2)$$

$$\mathcal{L}\{f\}(s) = \int_{0^-}^{\infty} f(t)e^{-st} \, dt, \quad \mathcal{L}\{f\}(s) = \int_{0^-}^{\infty} f(t)e^{-st} \, dt$$

$$\mathcal{F}(f(x+x_0)) = \mathcal{F}(f(x))e^{2\pi i \xi x_0}, \quad \mathcal{F}(f(x+x_0)) = \mathcal{F}(f(x))e^{2\pi i \xi x_0}$$

mathtext: $A, F, L, 2, 3, 5, \sigma$, mathnormal: $A, F, L, 2, 3, 5, \sigma$, mathrm: $A, F, L, 2, 3, 5, \sigma$.

mathbf: $\mathbf{A}, \mathbf{F}, \mathbf{L}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \boldsymbol{\sigma}$, mathit: $A, F, L, 2, 3, 5, \sigma$, mathsf: $A, F, L, 2, 3, 5, \sigma$.

mathtt: $A, F, L, 2, 3, 5, \sigma$, mathfrak: $\mathfrak{A}, \mathfrak{F}, \mathfrak{L}, 2, 3, 5, \sigma$, mathbb: $\mathbb{A}, \mathbb{F}, \mathbb{L}, 2, 3, 5, \sigma$.

mathcal: $\mathcal{A}, \mathcal{F}, \mathcal{L}, 2, 3, 5, \sigma$, mathscr: $\mathscr{A}, \mathscr{F}, \mathscr{L}, 2, 3, 5, \sigma$, boldsymbol: $\mathbf{A}, \mathbf{F}, \mathbf{L}, 2, 3, 5, \boldsymbol{\sigma}$.

vector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitvector: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

matrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, unitmatrix: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

tensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$, untensor: $\boldsymbol{\sigma}, \mathbf{T}, \mathbf{a}, \mathbf{F}, \mathbf{n}$

15

致 谢

感激 casthesis 作者吴凌云学长, gbt7714-bibtex-style 开发者 zepinglee, 和 ctex 众多开发者们。若没有他们的辛勤付出和非凡工作, $\text{\LaTeX}$ 菜鸟的我无法完成此国科大学位论文 $\text{\LaTeX}$ 模板 ucasthesis 的。在 $\text{\LaTeX}$ 中的一点一滴的成长源于开源社区的众多优秀资料和教程, 在此对所有 $\text{\LaTeX}$ 社区的贡献者表示感谢!

作者简历及攻读学位期间发表的学术论文与研究成果

本科生无需此部分。

作者简历：

已发表（或正式接受）的学术论文：

申请或已获得的专利：

参加的研究项目及获奖情况：

[plain]